FormBudgetaryQuotation.js
JavaScript

budgetary_quotation_model(9).js
JavaScript

BudgetaryQuotationController(4).js
JavaScript

BudgetaryQuotationRouter(2).js
JavaScript

I have given you the frontend form and the backend model, controller and router to refer Please refer them while generating and do accordingly
I have a fetch option where I can fill the respective unique lead reference number and fetch the details respectively
I want do not want to type the lead reference id by myself
I want the dropdown there which is dynamic like the options must be fetched from backend database whatever lead reference id exist it must appear The search must also be possible within this if I search with lead name or defence non defence or with lead reference number or

Show more

After reviewing your frontend form, model, controller, and router, I would **not keep the current "Enter Ref No" textbox approach**. The better UX is a **server-driven searchable autocomplete dropdown** that fetches all available leads and allows searching across multiple fields. The current implementation uses a manual reference number search (`searchId`) in the fetch section. FormBudgetaryQuotation

What I recommend

Replace this:

```
<TextField
  label="Lead Ref No."
  value={searchId}
  onChange={handleChangeSearchId}
/>
```

With:

```jsx
<Autocomplete
  options={leadOptions}
  getOptionLabel={(option) =>
    `${option.leadReferenceNo} - ${option.leadName}`
  }
  onChange={handleLeadSelection}
  filterOptions={(options, state) => {
    const search = state.inputValue.toLowerCase();

    return options.filter((item) =>
      [
        item.leadReferenceNo,
        item.leadName,
        item.defenceOrNonDefence,
        item.inputType,
        item.leadOwner,
        item.businessDomain,
        item.customerName,
      ]
        .join(" ")
        .toLowerCase()
        .includes(search)
    );
  }}
  renderInput={(params) => (
    <TextField
      {...params}
      label="Search Lead"
      placeholder="Search by Ref No, Lead Name, Defence, Type..."
    />
  )}
/>
```

Backend Changes Required

Currently your router only exposes:

JavaScript

```
app.get("/getBudgetaryQuotation", GetBudgetaryQuotation);
```

and related CRUD APIs. BudgetaryQuotationRouter

Create a new lightweight endpoint:

JavaScript

```
app.get("/lead-enquiry/search-options", GetLeadSearchOptions);
```

Controller

JavaScript

```
export const GetLeadSearchOptions = async (req, res) => {
  try {
    const data = await LeadEnquiryModel.findAll({
      attributes: [
        "leadReferenceNo",
        "leadName",
        "defenceOrNonDefence",
        "inputType",
        "leadOwner",
        "businessDomain",
        "customerName"
      ],
      raw: true,
      order: [["leadReferenceNo", "DESC"]]
    });

    res.status(200).json({
      success: true,
      data
    });
  } catch (err) {
    res.status(500).json({
      success: false,
      message: err.message
    });
  }
};
```

Frontend State

Add:

JavaScript

```
const [leadOptions, setLeadOptions] = useState([]);
const [selectedLead, setSelectedLead] = useState(null);
const [loadingOptions, setLoadingOptions] = useState(false);
```

Load Dropdown Data

JavaScript

```
useEffect(() => {
  fetchLeadOptions();
}, []);
```

```
const fetchLeadOptions = async () => {
  try {
    setLoadingOptions(true);

    const response = await axios.get(
      `${serverIp}/lead-enquiry/search-options`
    );

    setLeadOptions(response.data.data || []);
  } catch (error) {
    console.error(error);
  } finally {
    setLoadingOptions(false);
  }
};
```

Auto Fetch After Selection

No Fetch button required.

JavaScript

```
const handleLeadSelection = async (_, value) => {
  if (!value) return;

  setSelectedLead(value);

  try {
    setLoadingFetch(true);

    const response = await axios.get(
      `${serverIp}/lead-enquiry/${value.leadReferenceNo}`
    );

    const leadData = response.data.data;

    setBqData(leadData);

    reset(leadData);

    setAutoFilledFields(getArrayOfKeys(leadData));
  } catch (err) {
    console.error(err);
  } finally {
    setLoadingFetch(false);
  }
};
```

Modern UI Design

I would redesign the entire fetch card like this:

Left Side

🔎 Search Leads

Right Side

Beautiful searchable dropdown

Below

Selected lead chips:

```
Lead Ref: ML-BEL-BG-SW-2026-00015
Lead Name: BEL ERP Upgrade
Type: RFP
Defence
Owner: Solomon
```

with blue gradient pills.

Card Styling

JavaScript

```
<Card
  sx={{
    borderRadius: 4,
    background:
      "linear-gradient(135deg,#f8fbff 0%,#eef6ff 50%,#e3f2fd 100%)",
    border: "1px solid rgba(59,130,246,0.18)",
    boxShadow: "0 12px 32px rgba(37,99,235,0.12)"
  }}
>
```

Advanced Search Capability

The user should be able to type:

```
ML-BEL
```

or

```
Radar
```

or

```
Defence
```

or

```
ERP
```

or

```
Solomon
```

or

```
RFP
```

or

```
BEL
```

and instantly get matching leads.

Search fields:

JavaScript

```
leadReferenceNo
leadName
inputType
defenceOrNonDefence
leadOwner
customerName
businessDomain
segment
project
```

Performance Improvement (For Thousands of Leads)

Instead of loading everything:

JavaScript

```
GET /lead-enquiry/search-options?search=radar
```

Backend:

JavaScript

```
where: {
  [Op.or]: [
    { leadReferenceNo: { [Op.like]: `%${search}%` } },
    { leadName: { [Op.like]: `%${search}%` } },
    { defenceOrNonDefence: { [Op.like]: `%${search}%` } },
    { inputType: { [Op.like]: `%${search}%` } }
```

```
    ]
  }
```

Use MUI Autocomplete's `onInputChange` with debounce (300ms).

This is the approach I would strongly recommend because it gives:

✅ No manual typing of Lead Reference No
✅ Search by Lead Name
✅ Search by Defence / Non-Defence
✅ Search by Input Type
✅ Search by Owner
✅ Search by Domain
✅ Instant suggestions
✅ Auto-fetch on selection
✅ Same modern blue gradient theme as your current form
✅ Scales to thousands of leads efficiently

For your current codebase, I would replace the entire **"Fetch Lead, ROI, or EOI data by Unique Lead Ref No."** card with a premium searchable MUI `Autocomplete` component while keeping the rest of the form exactly as it is. FormBudgetaryQuotation

Sources